

Übung zur Vorlesung Grundlagen Betriebssysteme und Rechnernetze

5. Übungsblatt, Abgabe bis Mo, 12. Januar 2026, 10:00 Uhr

Hinweise zur Bearbeitung und Abgabe der Übung:

- Die Übungsaufgaben und Zusatzmaterial finden Sie auf Moodle:
<https://moodle2.uni-potsdam.de/course/view.php?id=47479>
- Abgabe in 3er-Gruppen; tragen Sie sich im Moodle-Kurs in eine Abgabegruppe ein! Auf allen Abgaben Namen und Matrikelnummern der Gruppenmitglieder nicht vergessen!
- Laden Sie Ihre Lösungen der Theorieaufgaben als PDF in Moodle hoch.
- Nutzen Sie für die Praxisaufgaben Git.UP! Klonen Sie sich für die jeweilige Aufgabe die Vorlage aus der Gruppe **GBR Vorlagen**. Sollte es für die Aufgabe keine Vorlage geben, zB: Aufgabe 1.1, erstellen Sie ein eigenes Repository. Achten Sie darauf ihre Git-Projekte im GBR Namespace zu erstellen. Für weitere Details folgen sie dem Manual auf Moodle.
- Halten Sie sich an die in den praktischen Aufgaben gegebenen Dateinamen und Methodennamen!
- Bitte keine Umlaute und Leerzeichen in Verzeichnissen und Dateinamen in der Abgabe verwenden! Ersetzen Sie Umlaute in Dateinamen (ä wird zu ae etc.)!
- Bei Problemen mit den Praxisaufgaben oder den Abgaben in GitLab legen Sie bitte in Ihrem GitLab-Repository ein Issue an und markieren darin einen Lehrenden! Bei Fragen zu den Aufgaben benutzen Sie bitte das Diskussionsforum in Moodle!
- Bitte kommentieren Sie Ihre Programme sinnvoll! Verwenden Sie sprechende Bezeichner und behandeln Sie mögliche Fehler!
- Stellen Sie sicher, dass Ihre Programme auf `tiree.lab.cs.uni-potsdam.de` compilierbar und lauffähig sind!

Gesamtpunktzahl des Aufgabenblattes: 20 Punkte.

Aufgabe 5.1: POSIX-Threads, Reader/Writer Locks

- a) Auf GitUP finden Sie eine Implementierung einer doppelt verketteten Liste. Erweitern Sie die Implementation, sodass Sie von mehreren Threads gleichzeitig genutzt werden kann. Realisieren Sie dies mit den Read-Write-Locks `pthread_rwlock_rdlock()` bzw. `pthread_rwlock_wrlock()` aus der POSIX-Thread-Bibliothek! Ändern Sie dabei nicht die Funktionssignaturen in der Datei `list.c`.
- b) Schreiben Sie eine Testapplikation (`test_all.c`), die Ihre doppelt verkettete Liste initialisiert und mit Testwerten befüllt. Anschließend sollen 3 Threads erstellt werden, die jeweils die insert-, delete- und get-Methode mehrfach ausführen. Danach soll die doppelt verkettete Liste wieder freigegeben werden. Jeder Thread arbeitet eine eigene Eingabefolge ab.

(4 + 2 Punkte)

Aufgabe 5.2: Synchronisation

- a) Worin besteht der Unterschied zwischen dem Java- Monitor- und dem Hoare-Monitorkonzept?
- b) Die folgende Umsetzung der put- und get-Funktionen des Java-Cubbyhole-Beispiels ist fehlerhaft. Wie kann man den Fehler beheben?

```
public synchronized int get() {
    if (available == false) {
        try {
            wait();
        } catch (InterruptedException e) {
        }
    }
    available = false;
    notify();
    return contents;
}

public synchronized void put(int value) {
    if (available == true) {
        try {
            wait();
        } catch (InterruptedException e) {
        }
    }
    contents = value;
    available = true;
    notify();
}
```

- c) Überdies kann es auch in der korrigierten Lösung zu Livelocks kommen. Geben Sie ein Beispielszenario dafür an! (1 + 1 + 1 Punkte)

Aufgabe 5.3: Buddy-System

Ein Betriebssystem verwende das Buddy-System für die Speicherverwaltung mit 2^k KB ($k \geq 0$) großen Blöcken. Der Rechner ist mit 2048 KByte RAM ausgestattet, die vollständig im Buddy-System verwaltet werden.

Die Prozesse A, B, C und D fordern Speicher in folgender Reihenfolge an:

- i) A fordert 236 KByte an.
 - ii) B fordert 144 KByte an.
 - iii) C fordert 76 KByte an.
 - iv) D fordert 456 KByte an.
- a) Zeichnen Sie, nach den Schritten ii) , iii) und iv) die Speicherbelegung ausgehend vom komplett freien und nicht in Blöcke eingeteilten Speicher.
 - b) Wieviele Blöcke sind noch übrig? Wie groß ist dann der größte verfügbare Speicherbereich?
 - c) Geben Sie den aktuellen Verschnitt an!

Nacheinander geben dann die Prozesse B, A, D und C in dieser Reihenfolge ihren Speicher frei.

- d) Zeichnen Sie, ausgehend vom Ergebnis von a), für jeden Schritt die Speicherbelegung. Geben Sie nach jeder Speicherfreigabe den größten zur Verfügung stehenden Block an!

(1 + 0,5 + 0,5 + 1 + 1 Punkte)

Aufgabe 5.4: Gutartigkeit

Zeigen Sie, daß LRU ein gutartiger Seitenersetzungsalgorithmus ist! (2 Punkte)

Aufgabe 5.5: Speicherverwaltung Seitenfehler

Die Behandlung eines Seitenfehlers erfordert i. A. Maßnahmen zur Ersetzung einer Seite in einem Seitenrahmen (Kachel) des Arbeitsspeichers. Dazu muss zunächst eine andere Seite aus dem Arbeitsspeicher verdrängt werden.

Bei der Ausführung eines Benutzerprogramms tritt folgende Sequenz von Seitenzugriffen auf:

7 6 2 7 5 5 3 2 7 4 5

Bestimmen Sie die minimale Anzahl der Seitenfehler, die auftreten kann! Gehen Sie dabei von 4 Seitenrahmen aus und begründen Sie Ihre Antwort. (2 Punkte)

Aufgabe 5.6: Speicherverwaltung Algorithmen

Es steht ein physikalischer Adressraum mit 4 Seitenrahmen zur Verfügung. Arbeiten Sie die angegebene Seitenzugriffsfolge gemäß der Seitenersetzungsstrategien:

- FIFO (First-In-First-Out),
- LRU (Least-Recently-Used),
- Auslagern einer Seite, die nicht zum Working-Set gehört (Working-Set-Parameter $T=3$).

ab, und vergleichen Sie diese anhand der Seitenfehleranzahl!

2 7 6 7 1 4 7 2 5 5 1

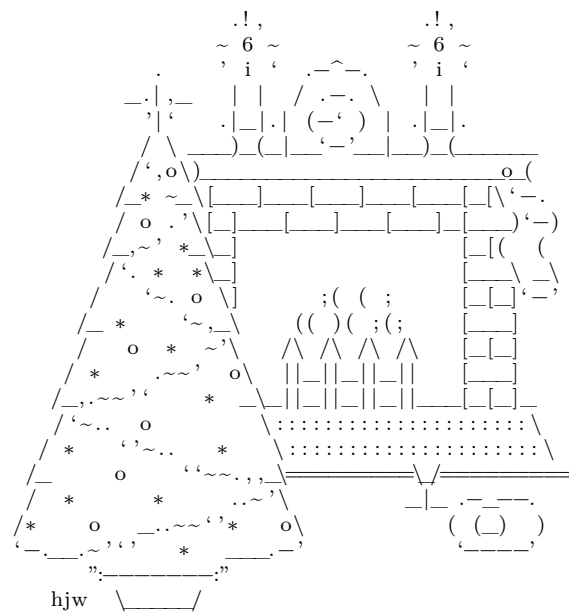
In denjenigen Fällen, in denen Nicht-Determiniertheit vorliegt, ist stets die Seite mit der kleinsten Seitenrahmennummer zu wählen, die das jeweilige Verfahren erfüllt.

(3 Punkte)

Aufgabe 5.7: ASCII-Art

ASCII-Art ist nicht nur zur Weihnachtszeit eine bezaubernde Kunstform. Schreiben Sie ein C-Programm, welches ein Bild passend zur Weihnachtszeit oder zum Jahreswechsel auf der Konsole, nur bestehend aus ASCII Zeichen, ausgibt. Zusätzlich zu den druckbaren ASCII Zeichen sind auch Steuerzeichen erlaubt, welche den Status der Ausgabe, z.B. die Farbe ändern. Eine musikalische Untermalung durch ASCII Code 0x07 sorgt in der kalten Jahreszeit für zusätzliche Freude.

(4 Zusatzpunkte)



Quelle: <https://ascii.co.uk/art/christmas>